

IFeatureRenderer

Interface Contract

Rendering Engine Integration Handoff — WP3 / Sub-Team 5

Document type	Technical Interface Contract
Version	1.0
Date	2026-05-28
Issuing team	Sub-Team 5 — Feature System & Domain Model
Target team	WP3 — Rendering Engine
Classification	Internal / Confidential

Contents

§	Section	Page
1	Background & Purpose	3
2	What Team 5 Has Changed	4
3	The IFeatureRenderer Contract	5
4	The Dirty-Notification Chain	7
5	Step-by-Step Implementation Plan for WP3	8
6	Before / After Code Reference	12
7	Responsibility Summary	14
8	Acceptance Criteria	15

1. Background & Purpose

This document is issued by **Sub-Team 5 (Feature System & Domain Model)** to **WP3 (Rendering Engine)** as part of the iDaVIE refactoring assignment. It defines the agreed interface boundary between the pure-C# domain layer and the GPU rendering layer, specifies exactly what WP3 must change in **FeatureSetRenderer.cs** to conform to that boundary, and provides the acceptance criteria that must pass before integration is considered complete.

Prior to the refactoring, **FeatureSetRenderer** acted as both the domain container for Feature objects (maintaining the authoritative list, managing parent wiring, loading VOTable data) *and* the GPU renderer (managing ComputeBuffers, vertex pulling, OnRenderObject). These two responsibilities were tightly entangled, making it impossible to unit-test domain logic without a live Unity runtime, impossible to swap the renderer, and difficult for two sub-teams to work on the same file simultaneously.

Sub-Team 5 has completed the domain-side refactoring. The domain is now a clean, Unity-free set of C# classes (**Feature**, **FeatureSet**, **FeatureCatalog**, **FeatureSetService**) that compile and unit-test without Unity. The **FeatureVisualiser** MonoBehaviour acts as the sole adapter between the domain and Unity. WP3 retains ownership of the concrete GPU implementation; this document tells WP3 exactly how to align that implementation with the new domain boundary.

Scope of this document

This contract covers only the interface between the domain layer and the rendering layer. It does **not** cover VOTable I/O (WP2/WP7 boundary), the statistics provider boundary (DataAnalysis DLL), or VR input handling.

2. What Sub-Team 5 Has Changed

The following changes have already been committed to the **DataFeatures** namespace. WP3 does **not** need to replicate these; they are listed here so WP3 understands why their existing code will no longer compile and what the intended architecture looks like.

2.1 Feature.cs — FeatureSetParent type change

The property **FeatureSetParent** has been changed from **IFeatureDirtyNotifier** (a minimal dirty-notification interface) to **FeatureSet** (the full domain aggregate). The setter is now **private**. External code — including **FeatureSetRenderer** — can no longer assign to this property directly.

Two explicit methods are now exposed instead:

```
// Feature.cs (Team 5 – already committed)
public FeatureSet FeatureSetParent { get; private set; }

public void SetParent(FeatureSet parent) => FeatureSetParent = parent;
public void ClearParent()                => FeatureSetParent = null;
```

Feature.cs — new parent wiring API

All dirty notifications inside **Feature** now route through **FeatureSetParent?.NotifyDirty(Index)**, which fires the **FeatureSet.FeatureDirty** event rather than calling directly into the renderer.

2.2 FeatureSet.cs — Add / Remove own parent wiring

FeatureSet.Add(feature) calls **feature.SetParent(this)** before appending to the internal list, and fires **FeatureDirty?.Invoke(feature.Index)**. **FeatureSet.Remove(feature)** calls **feature.ClearParent()** and fires **FeatureDirty?.Invoke(-1)** (full redraw). WP3 must use **FeatureSet.Add()** when building the feature list rather than appending to the renderer's internal **FeatureList** directly.

2.3 FeatureVisualiser.cs — Renderer registry & anchor fix

FeatureVisualiser now maintains a **Dictionary<FeatureSet, FeatureSetRenderer> _setRenderers**. Every call to **CreateRendererForSet(set)** registers the renderer in this dictionary and subscribes **set.FeatureDirty += iRenderer.SetFeatureAsDirty**. The **RepositionAnchors** method now looks up the renderer's transform through this dictionary rather than casting through **IFeatureDirtyNotifier**, eliminating the previous compile error.

■ Warning

Because FeatureSetParent now has a private setter, any line in FeatureSetRenderer.cs that reads **featureToAdd.FeatureSetParent = this** will produce a CS0272 compile error. WP3 must remove these lines as described in Section 5.

3. The IFeatureRenderer Contract

IFeatureRenderer is defined in **Assets/Scripts/FeatureData/IFeatureRenderer.cs** and is owned by Sub-Team 5. WP3 provides the concrete implementation in **FeatureSetRenderer.cs**. Neither team should modify the interface without written agreement from both sides.

3.1 Full interface definition

```
namespace DataFeatures
{
    /// <summary>
    /// Rendering boundary for a single FeatureSet.
    /// Implemented by FeatureSetRenderer (WP3 - GPU rendering layer).
    /// Called only by FeatureVisualiser - never by domain objects.
    /// </summary>
    public interface IFeatureRenderer
    {
        /// <summary>Adds a feature so it is included in the next draw call.</summary>
        void AddFeature(Feature feature);

        /// <summary>Removes a feature from the render list.</summary>
        void RemoveFeature(Feature feature);

        /// <summary>Clears all features from the render list.</summary>
        void ClearFeatures();

        /// <summary>
        /// Marks a feature as needing a vertex-buffer update on the next frame.
        /// Pass -1 to mark all features dirty.
        /// Subscribed directly to FeatureSet.FeatureDirty.
        /// </summary>
        void SetFeatureAsDirty(int index);

        /// <summary>
        /// Display colour for all features in this set.
        /// Uses the domain FeatureColor struct - no Unity dependency.
        /// Implementor converts to UnityEngine.Color internally.
        /// </summary>
        FeatureColor FeatureColor { get; set; }
    }
}
```

IFeatureRenderer.cs — owned by Sub-Team 5, implemented by WP3

3.2 Method-by-method specification

Method / Property	Signature	Contract & Constraints
AddFeature	<code>void AddFeature(Feature feature)</code>	Appends feature to the renderer's internal GPU list. Must NOT set <code>feature.FeatureSetParent</code> (domain's responsibility). Must call <code>SetFeatureAsDirty</code> for the new feature's index so it is drawn on the next frame. Safe to call on an already-present feature (no-op or replace, implementor's choice).
RemoveFeature	<code>void RemoveFeature(Feature feature)</code>	Removes feature from the GPU list. After removal, calls <code>SetFeatureAsDirty(-1)</code> to force a full buffer rebuild. Must NOT call <code>feature.ClearParent()</code> — that is the domain's responsibility via <code>FeatureSet.Remove()</code> .
ClearFeatures	<code>void ClearFeatures()</code>	Empties the GPU list entirely. Equivalent to calling <code>SetFeatureAsDirty(-1)</code> after clearing. Used when a <code>FeatureSet</code> is deleted or reassigned.
SetFeatureAsDirty	<code>void SetFeatureAsDirty(int index)</code>	Schedules a GPU vertex-buffer update for the given index on the next <code>Update()</code> frame. <code>index == -1</code> means rebuild all vertices. This method is subscribed directly to <code>FeatureSet.FeatureDirty</code> by <code>FeatureVisualiser</code> — WP3 must never subscribe or unsubscribe from that event directly.
FeatureColor	<code>FeatureColor FeatureColor { get; set; }</code>	Gets/sets the uniform tint for all features in this set. Uses <code>DataFeatures.FeatureColor</code> (a Unity-free struct with R, G, B, A floats). The implementor converts to <code>UnityEngine.Color</code> internally. The existing explicit interface implementation in <code>FeatureSetRenderer</code> is correct and does not need to change.

4. The Dirty-Notification Chain

Understanding this chain is critical. The refactoring eliminates the direct coupling where Feature called methods on FeatureSetRenderer. Every state change now travels through the domain aggregate before reaching the renderer.

4.1 Full chain — state change to GPU update

Step	Location	What happens
1	Feature.cs (property setter)	User code sets feature.Selected = true (or .Visible, .CubeColor, .Center, etc.)
2	Feature.cs NotifyDirty()	Private NotifyDirty() calls FeatureSetParent?.NotifyDirty(Index)
3	FeatureSet.cs NotifyDirty(int)	internal NotifyDirty fires the public FeatureDirty event with the index
4	FeatureVisualiser.cs CreateRendererForSet()	set.FeatureDirty += iRenderer.SetFeatureAsDirty (wired here at construction time)
5	FeatureSetRenderer.cs SetFeatureAsDirty(int)	Adds index to _dirtyFeatures list (or clears + adds -1 if index == -1)
6	FeatureSetRenderer.cs Update()	On next frame: rebuilds ComputeBuffer vertices for dirty indices only
7	FeatureSetRenderer.cs OnRenderObject()	GPU draws updated buffer via Graphics.DrawProceduralNow

Steps 1 - 3 are domain-only (no Unity, fully unit-testable).

Steps 5 - 7 are renderer-only (pure GPU work, owned by WP3). Step 4 is the wiring performed once by FeatureVisualiser — neither team should replicate or modify this subscription.

4.2 Two independent renderer operations

There are **two distinct operations** that look similar but serve different purposes. WP3 must keep them separate:

IFeatureRenderer.AddFeature(feature)	IFeatureRenderer.SetFeatureAsDirty(index)
Adds a Feature object to the renderer's internal GPU list (FeatureList). Called by FeatureVisualiser once per feature when the renderer is first created, and again whenever a new feature is loaded into the FeatureSet after construction. This operation grows the renderer's list.	Marks an existing entry in FeatureList as needing a vertex-buffer refresh on the next frame. Does NOT add or remove from the list. Called automatically by the FeatureDirty event subscription. This operation refreshes GPU data for existing entries.

5. Step-by-Step Implementation Plan for WP3

All five steps below must be completed before integration. Steps 1 and 2 fix immediate compile errors. Steps 3 through 5 ensure the domain and renderer stay in sync at runtime. Work through them in order — each step builds on the previous.

1 Remove FeatureSetParent = this from AddFeature()

Location: **FeatureSetRenderer.cs**, inside the **AddFeature(Feature featureToAdd)** method.

The existing implementation sets `featureToAdd.FeatureSetParent = this` immediately after appending the feature to `FeatureList`. This line must be deleted. Since `FeatureSetParent` now has a private setter, this line is a CS0272 compile error and will prevent the project from building.

After deletion, **AddFeature** should only do renderer work: append to `FeatureList`, update the menu scroller, and call `SetFeatureAsDirty(featureToAdd.Index)`. The parent wiring is now handled automatically by **FeatureSet.Add()** on the domain side — the renderer must not touch it.

2 Remove FeatureSetParent = this from SpawnFeaturesFromTable()

Location: **FeatureSetRenderer.cs**, inside the inner for loop of **SpawnFeaturesFromTable()**, at the point where each `Feature` is constructed and conditionally added.

There is a second `featureToAdd.FeatureSetParent = this` assignment here — it must also be deleted for the same reason. This method constructs `Feature` objects and appends them to the renderer's `FeatureList` directly. After removing the parent assignment, the renderer-side list population can remain, but see Step 3 for the domain-side change that must accompany it.

3 Thread a FeatureSet parameter through SpawnFeaturesFromTable() and SpawnFeaturesFromSourceStats()

Location: **FeatureSetRenderer.cs** — both spawn methods, and their call sites in **FeatureVisualiser.cs**.

Currently both `SpawnFeaturesFromTable` and `SpawnFeaturesFromSourceStats` build `Feature` objects and add them only to the renderer's internal `FeatureList`. This leaves the domain `FeatureSet` empty, so the domain layer has no knowledge of those features and cannot reason about them (selection, export, stats, etc.).

The fix has two parts that must happen together in each spawn method:

1. Accept a FeatureSet parameter

Add a FeatureSet set parameter to both method signatures. The caller (FeatureVisualiser) already creates the FeatureSet before calling the spawn method, so passing it through is a one-line change at each call site. The updated signatures become: SpawnFeaturesFromTable(mapping, table, set, columnsMask, excludeExternal) and SpawnFeaturesFromSourceStats(sourceStatsDict, set).

2. Call set.Add(feature) for each constructed Feature

For every Feature that would have been added to the renderer's list, call set.Add(featureToAdd) first. This call handles parent wiring and fires FeatureDirty. Then also call this.AddFeature(featureToAdd) (or FeatureList.Add plus SetFeatureAsDirty) to populate the GPU list. Both must happen because the domain list and the GPU list are independent: the domain list is the source of truth for feature data; the GPU list is the renderer's working copy for vertex generation.

4 Verify AddFeature() touches no domain state

Location: **FeatureSetRenderer.cs** — **AddFeature()**, reviewed in its entirety.

After Steps 1 and 3, do a final audit of AddFeature(). The method must contain **only** the following categories of operation: appending to FeatureList; updating the FeatureMenuScrollerDataSource; calling SetFeatureAsDirty(). It must not read or write any property on Feature that belongs to the domain (e.g., FeatureSetParent, Index reassignment, RawData modification). The renderer is a consumer of Feature data, not a mutator of domain state.

If any domain mutations remain in AddFeature(), move them either to FeatureSet.Add() (domain side, via team 5) or to the FeatureVisualiser adapter layer, depending on which team owns the concern.

5 Confirm IFeatureRenderer.FeatureColor explicit implementation still compiles

Location: **FeatureSetRenderer.cs** — the explicit interface property at the bottom of the class.

The existing explicit implementation bridges DataFeatures.FeatureColor (a Unity-free struct with R, G, B, A floats defined in FeatureSet.cs) to the internal UnityEngine.Color field. This implementation is architecturally correct and does not need to change. However, after Steps 1-4 have been applied, perform a full project build to confirm this property compiles cleanly and no residual reference to the old IFeatureDirtyNotifier interface exists anywhere in FeatureSetRenderer.cs.

If the build still references IFeatureDirtyNotifier in any way (e.g., a cast, an is check, or an as expression), remove it. That interface is now considered internal to the domain layer and may be removed or made internal in a future cleanup pass. WP3 code must not depend on it.

6. Before / After Code Reference

The following side-by-side comparisons show exactly what needs to change in FeatureSetRenderer.cs. Left column is the current (broken) code; right column is the required (correct) code. Only the relevant excerpts are shown.

6.1 AddFeature()

BEFORE (current — compile error)	AFTER (required)
<pre>public void AddFeature(Feature featureToAdd) { FeatureList.Add(featureToAdd); FeatureMenuItemInfo obj = new ...; obj.IdTextField = (FeatureList.Count)...; obj.SourceName = featureToAdd.Name; obj.Feature = featureToAdd; featureToAdd.FeatureSetParent = this; SetFeatureAsDirty(featureToAdd.Index); }</pre>	<pre>public void AddFeature(Feature featureToAdd) { FeatureList.Add(featureToAdd); FeatureMenuItemInfo obj = new ...; obj.IdTextField = (FeatureList.Count)...; obj.SourceName = featureToAdd.Name; obj.Feature = featureToAdd; // line removed - domain handles wiring SetFeatureAsDirty(featureToAdd.Index); }</pre>

6.2 SpawnFeaturesFromTable() — inner loop

BEFORE (current — compile error + domain bypass)	AFTER (required)
<pre>var featureToAdd = new Feature(...); featureToAdd.FeatureSetParent = this; if (!(excludeExternal && FeatureIsWithinVolume(...))) { FeatureList.Add(featureToAdd); }</pre>	<pre>var featureToAdd = new Feature(...); // no FeatureSetParent assignment if (!(excludeExternal && FeatureIsWithinVolume(...))) { set.Add(featureToAdd); // domain wiring AddFeature(featureToAdd); // GPU list }</pre>

7. Responsibility Summary

The table below captures the complete division of ownership between Sub-Team 5 and WP3. Items marked DONE are already committed. Items marked WP3 TODO are the outstanding work described in Section 5.

Concern	Owner	Mechanism	Status
Feature parent wiring (Feature.FeatureSetParent)	Sub-Team 5	FeatureSet.Add() calls feature.SetParent(this)	DONE
Feature parent clearing (on removal)	Sub-Team 5	FeatureSet.Remove() calls feature.ClearParent()	DONE
Dirty notification from Feature to domain aggregate	Sub-Team 5	Feature.NotifyDirty() calls FeatureSet.NotifyDirty(index)	DONE
Dirty event subscription (domain -> renderer)	FeatureVisualiser (adapter)	set.FeatureDirty += iRenderer.SetFeatureAsDirty	DONE
GPU vertex-buffer rebuild (per dirty index)	WP3	FeatureSetRenderer.Update() reads _dirtyFeatures list	DONE (no change needed)
GPU list population (AddFeature)	WP3	IFeatureRenderer.AddFeature() appends to FeatureList	WP3 TODO (remove domain mutation)
Spawn features from VOTable (domain + GPU sync)	WP3 calls Sub-Team 5 API	set.Add() for domain; AddFeature() for GPU	WP3 TODO (Steps 2 + 3)
FeatureColor conversion (domain struct -> Unity Color)	WP3	Explicit IFeatureRenderer implementation in renderer	DONE (no change needed)
Anchor transform lookup (RepositionAnchors)	Sub-Team 5	_setRenderers dictionary in FeatureVisualiser	DONE
IFeatureRenderer interface definition	Sub-Team 5	IFeatureRenderer.cs — do not modify without agreement	DONE (locked)

8. Acceptance Criteria

Integration is considered complete when all of the following criteria pass. Sub-Team 5 will review WP3's changes against these criteria before the interface boundary is signed off.

#	Criterion	How to verify
AC-1	Project builds with zero CS0272 errors (FeatureSetParentInvalidSetViolation) of the DataFeatures assembly. Zero errors expected.	Build project and check for CS0272 errors (FeatureSetParentInvalidSetViolation) of the DataFeatures assembly. Zero errors expected.
AC-2	No reference to IFeatureDirtyNotifier exists in FeatureSetRenderer.cs. FeatureSetRenderer.cs should return no results.	Search for IFeatureDirtyNotifier in FeatureSetRenderer.cs. No results should be returned.
AC-3	No FeatureSetParent assignment exists in FeatureSetRenderer.cs. FeatureSetRenderer.cs should return no results.	Search for FeatureSetParent in FeatureSetRenderer.cs. No results should be returned.
AC-4	After SpawnFeaturesFromTable completes, FeatureSetRenderer.Count equals FeatureSetParent.Count. Both counts should be equal.	Run SpawnFeaturesFromTable and then compare FeatureSetRenderer.Count with FeatureSetParent.Count. Both counts should be equal.
AC-5	Modifying feature.Selected, feature.Visible, or feature.ParentColor triggers a PropertyChanged event. PropertyChanged event should be raised.	Modify feature.Selected, feature.Visible, or feature.ParentColor and verify that a PropertyChanged event is raised.
AC-6	FeatureSetRenderer.AddFeature() does not read or modify FeatureSetParent.Count. FeatureSetParent.Count should remain unchanged.	Call AddFeature() and verify that FeatureSetParent.Count remains unchanged.
AC-7	FeatureVisualiser.CreateRendererForSet() correctly instantiates FeatureSetRenderer. FeatureSetRenderer should be created.	Call CreateRendererForSet() and verify that a FeatureSetRenderer object is created.
AC-8	Anchor handles reposition correctly when a feature is selected. (Repositioning verified by Sub-Team 5 on a separate project)	Select a feature and verify that the Anchor handles repositioning correctly.

Sign-off process

Once WP3 considers all five steps complete, raise a pull request against the main integration branch and request a review from the Sub-Team 5 lead. The PR description must include a checklist confirming AC-1 through AC-8. Sub-Team 5 will conduct a code review and run the play-mode acceptance tests before approving the merge.